

Core avanzado: Threading, manejo de archivos y serialización

La combinación de concurrencia (threading), manejo avanzado de archivos y serialización es un patrón clásico en backends de Java. Un ejemplo representativo es un sistema de procesamiento por lotes (*batch processing*), donde múltiples hilos procesan transacciones en paralelo, actualizan contadores compartidos de forma segura, y finalmente serializan y guardan un reporte consolidado en disco.

Ejemplo Completo Integrado

El siguiente código implementa un flujo completo utilizando `ExecutorService`, manejo seguro de concurrencia con `AtomicInteger`, persistencia con `java.nio.file.Files` y serialización de objetos mediante `ObjectOutputStream` y `ObjectInputStream`.

```
5 // 1. Clase serializable con control de versión y campos transient
6 class ProcessingReport implements Serializable {
7     // Control de versión para la serialización
8     private static final long serialVersionUID = 1L;
9
10    private final String batchName;
11    private final int processedItemsCount;
12
13    // Este campo NO se guardará en disco (se ignorará en la serialización)
14    private final transient String internalSessionId;
15
16    public ProcessingReport(String batchName, int processedItemsCount, String internalSessionId) {
17        this.batchName = batchName;
18        this.processedItemsCount = processedItemsCount;
19        this.internalSessionId = internalSessionId;
20    }
21
22    public int getProcessedItemsCount() { return processedItemsCount; }
23    public String getInternalSessionId() { return internalSessionId; }
24
25    @Override
26    public String toString() {
27        return "ProcessingReport { batchName='" + batchName +
28            "', items=" + processedItemsCount +
29            ", internalSessionId='" + internalSessionId + "' }";
30    }
31 }
```

La clase **ProcessingReport** es un modelo de datos inmutable diseñado para representar y almacenar el resultado de un lote de procesamiento. Está preparada específicamente para ser persistida en disco gracias a la interfaz `Serializable`.

A continuación se detallan sus componentes principales:

1. Control de Versión para Serialización

- **serialVersionUID = 1L**: Define un identificador único de versión para la clase. Esto garantiza que al leer un objeto guardado previamente en disco, Java pueda verificar si la estructura de la clase coincide y evitar errores de compatibilidad (`InvalidClassException`).

2. Atributos de la Clase

- **batchName y processedItemsCount:** Almacenan el nombre del lote y la cantidad de elementos procesados con éxito. Al ser declarados como final, se garantiza su inmutabilidad una vez creados.
- **internalSessionId (con transient):** Este campo está marcado como transient, lo que le indica a Java que **debe ignorarlo** durante el proceso de serialización. Al guardar el objeto en disco, este valor no se escribe; por lo tanto, al recuperarlo, su valor será nulo (null). Se usa comúnmente para información sensible, temporal o datos que no necesitan persistencia.

3. Métodos y Comportamiento

- **Constructor:** Recibe y asigna los valores para los tres atributos principales al instanciar el objeto.
- **Getters:** Proveen acceso de lectura controlado a los campos privados (getProcessedItemsCount() y getInternalSessionId()).
- **toString():** Sobrescribe el método estándar para retornar una representación en texto limpia y legible del objeto y sus valores actuales (incluyendo el identificador de sesión).

```

12 public class AdvancedCoreDemo {
13     public static void main(String[] args) {
14         Path filePath = Paths.get("reporte_lote.ser");
15
16         // 2. Concurrencia: Contador seguro para evitar Race Conditions
17         AtomicInteger successCounter = new AtomicInteger(0);
18
19         // ExecutorService para gestionar un pool de hilos de forma eficiente
20         ExecutorService executor = Executors.newFixedThreadPool(4);
21         List<Callable<Void>> tasks = new ArrayList<>();
22
23         // Simulamos 10 tareas concurrentes de procesamiento
24         for (int i = 1; i <= 10; i++) {
25             final int taskId = i;
26             tasks.add(() -> {
27                 Thread.sleep(50); // Simulando tarea de trabajo
28
29                 // Incremento atómico y seguro entre múltiples hilos
30                 successCounter.incrementAndGet();
31                 System.out.println("Hilo [" + Thread.currentThread().getName() + "] procesó la tarea #" + taskId);
32                 return null;
33             });
34         }
35
36         try {
37             // Ejecutamos todas las tareas concurrentemente y esperamos a que terminen
38             executor.invokeAll(tasks);
39         } catch (InterruptedException e) {
40             Thread.currentThread().interrupt();
41             System.err.println("La ejecución concurrente fue interrumpida.");
42         } finally {
43             executor.shutdown(); // Liberamos los recursos del pool
44         }

```

El método main de la clase **AdvancedCoreDemo** actúa como el núcleo orquestador del programa, integrando la ejecución concurrente, el manejo seguro de datos compartidos, la persistencia en archivos y la serialización.

A continuación se detalla paso a paso lo que realiza este bloque de código:

1. Gestión de Concurrencia (Hilos y Sincronización)

- **Pool de Hilos (ExecutorService):** Se inicializa un pool fijo de 4 hilos (Executors.newFixedThreadPool(4)) para gestionar la ejecución eficiente de 10 tareas concurrentes.

- **Control de Race Conditions (AtomicInteger):** Se declara successCounter para llevar la cuenta de los elementos procesados con éxito. Dado que múltiples hilos intentarán modificar este valor simultáneamente, el uso de métodos atómicos como successCounter.incrementAndGet() previene la pérdida de actualizaciones sin necesidad de bloqueos pesados (synchronized).
- **Ejecución y Cierre:** Las tareas se agrupan en una lista y se ejecutan de manera concurrente mediante executor.invokeAll(tasks). El bloque finally asegura que el pool de hilos se libere correctamente con executor.shutdown() una vez finalizado el proceso.

```

45 |
46 | // Creamos el objeto con el resultado final consolidado por los hilos
47 | ProcessingReport report = new ProcessingReport("LOTE-2026-X", successCounter.get(), "SECURE-SESSION-999");
48 |
49 | System.out.println("\n--- Objeto original antes de serializar ---");
50 | System.out.println(report);
51 |
52 | // 3. Archivos y Serialización: Escritura usando try-with-resources y java.nio.file.Files
53 | try (ObjectOutputStream oos = new ObjectOutputStream(Files.newOutputStream(filePath))) {
54 |     oos.writeObject(report);
55 |     System.out.println("\n[Archivo] Reporte serializado guardado exitosamente en: " + filePath.toAbsolutePath());
56 | } catch (IOException e) {
57 |     System.err.println("Error al escribir el archivo: " + e.getMessage());
58 | }
59 |
60 | // 4. Lectura (Deserialización) del objeto desde disco
61 | System.out.println("\n--- Leyendo objeto desde disco (Deserialización) ---");
62 | try (ObjectInputStream ois = new ObjectInputStream(Files.newInputStream(filePath))) {
63 |     ProcessingReport loadedReport = (ProcessingReport) ois.readObject();
64 |     System.out.println(loadedReport);
65 |     // Nota: internalSessionId aparecerá como null debido a la palabra clave transient
66 | } catch (IOException | ClassNotFoundException e) {
67 |     System.err.println("Error al leer el archivo: " + e.getMessage());
68 | }
69 | }
70 |

```

2. Consolidación de Resultados

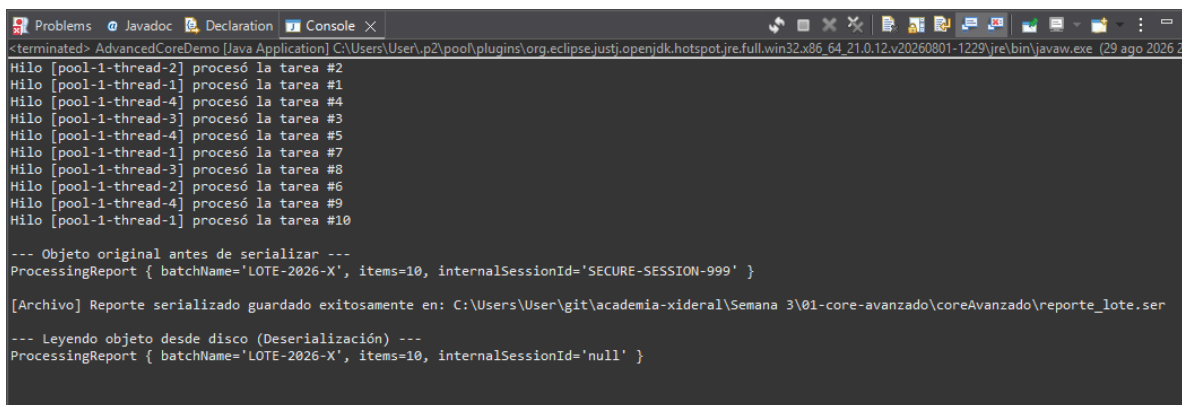
- Una vez que los hilos terminan su trabajo, se instancia el objeto ProcessingReport utilizando el total acumulado en el contador atómico (successCounter.get()), asignándole un nombre de lote ("LOTE-2026-X") y un ID de sesión interno.

3. Serialización y Escritura en Disco (java.nio.file.Files)

- Utilizando un bloque try-with-resources, se abre un flujo de salida conectando ObjectOutputStream con Files.newOutputStream(filePath).
- El objeto report se escribe directamente en el archivo binario reporte_lote.ser, garantizando el cierre automático de los recursos para evitar fugas de memoria o bloqueos de archivos.

4. Deserialización y Lectura desde Disco

- En la última sección, el programa vuelve a abrir el archivo reporte_lote.ser utilizando ObjectInputStream y Files.newInputStream(filePath).
- El flujo de bytes es leído y transformado nuevamente en un objeto Java (loadedReport), comprobando la recuperación exitosa de sus datos en consola (donde se evidencia que el campo transient se inicializa como null).



```
<terminated> AdvancedCoreDemo [Java Application] C:\Users\User\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.12.v20260801-1229\jre\bin\javaw.exe (29 ago 2026 2
Hilo [pool-1-thread-2] procesó la tarea #2
Hilo [pool-1-thread-1] procesó la tarea #1
Hilo [pool-1-thread-4] procesó la tarea #4
Hilo [pool-1-thread-3] procesó la tarea #3
Hilo [pool-1-thread-4] procesó la tarea #5
Hilo [pool-1-thread-1] procesó la tarea #7
Hilo [pool-1-thread-3] procesó la tarea #8
Hilo [pool-1-thread-2] procesó la tarea #6
Hilo [pool-1-thread-4] procesó la tarea #9
Hilo [pool-1-thread-1] procesó la tarea #10

--- Objeto original antes de serializar ---
ProcessingReport { batchName='LOTE-2026-X', items=10, internalSessionId='SECURE-SESSION-999' }

[Archivo] Reporte serializado guardado exitosamente en: C:\Users\User\git\academia-xideral\Semana 3\01-core-avanzado\coreAvanzado\reporte_lote.ser

--- Leyendo objeto desde disco (Deserialización) ---
ProcessingReport { batchName='LOTE-2026-X', items=10, internalSessionId='null' }
```

La imagen muestra el resultado de la ejecución exitosa del programa. En ella se plasma claramente el flujo integrado de concurrencia, archivos y serialización:

- **Procesamiento Concurrente (Hilos):** Las primeras líneas reflejan cómo un pool de hilos (pool-1-thread-1 hasta pool-1-thread-4) procesa de manera concurrente y asíncrona las 10 tareas.
- **Objeto Original:** Se muestra la representación en texto del objeto ProcessingReport antes de guardarlo, indicando que se completaron 10 elementos (items=10) para el lote LOTE-2026-X y mostrando su ID de sesión original (SECURE-SESSION-999).
- **Persistencia en Archivo:** Se imprime una confirmación con la ruta absoluta exacta en el equipo donde se guardó el archivo binario generado mediante serialización (reporte_lote.ser).
- **Lectura y Deserialización:** En la última línea se observa el objeto recuperado desde el disco. Aquí se evidencia de forma práctica el efecto de la palabra clave transient: el campo internalSessionId, que originalmente contenía texto, ahora aparece como null porque no fue almacenado en el archivo.

El problema de la concurrencia y su solución

- **¿Qué pasa si dos hilos tocan el mismo dato?** Si dos hilos intentan modificar una variable primitiva compartida (por ejemplo, counter++), ocurren condiciones de carrera (race conditions). La operación counter++ no es atómica: implica leer el valor actual, sumarle uno y escribirlo de nuevo. Si el Hilo A y el Hilo B leen el mismo valor al mismo tiempo, uno de los incrementos se pierde (pérdida de actualizaciones).
- **Cómo se resuelve:** En el ejemplo anterior se utilizó un AtomicInteger, el cual aprovecha instrucciones a nivel de hardware (como CAS - Compare-And-Swap) para asegurar que las operaciones de lectura y escritura sean seguras y atómicas sin bloquear explícitamente todo el hilo. Otras alternativas comunes son usar bloques synchronized o colecciones concurrentes (como ConcurrentHashMap).

¿Por qué existe el serialVersionUID?

El serialVersionUID es un identificador numérico único que actúa como una **firma de versión** para las clases que implementan Serializable.

- **Su propósito:** Garantizar la compatibilidad durante la deserialización. Cuando la Máquina Virtual de Java (JVM) lee un flujo de bytes para convertirlo en objeto, compara el serialVersionUID del archivo con el de la clase cargada actualmente en memoria.
- **Qué evita:** Si modificas la estructura de la clase (agregando o quitando campos) y no defines un serialVersionUID explícito, Java genera uno automáticamente basado en los atributos de la clase. Si intentas leer un archivo viejo con una clase modificada, la JVM lanzará un error de tipo InvalidClassException. Definirlo explícitamente (ej. 1L) permite controlar cuándo una clase es compatible a pesar de cambios menores.

¿Qué le pasa a un campo marcado con transient?

- **Comportamiento:** La palabra clave transient le indica al mecanismo de serialización que ignore por completo ese campo al momento de guardar el objeto en disco.
- **Resultado al deserializar:** Cuando el objeto es leído nuevamente desde el archivo, el campo transient no recupera su valor original, sino que se inicializa con su valor por defecto predeterminado en Java (null para objetos, 0 para numéricos y false para booleanos).
- **Casos de uso típicos:** Se utiliza para datos sensibles (contraseñas), información temporal de sesión, conexiones abiertas a bases de datos (Connection), hilos activos (Thread), o datos recalculables que no tiene sentido persistir.